

Actividad 1.4 Join Cafetería

Álvaro Del Valle Fernández

October 29, 2025

1 Introducción

En este documento hablaré sobre la estructura de mi proyecto de de Cafetería con Joins usando javafx, la idea es crear un proyecto en el cual dos camareros gestionen multiples clientes como threads, el elemento fundamental es el uso del Join en este ejercicio.

Este sistema crea una espera para sincronizar los hilos correctamente, en este ejercicio veremos como funciona junto a una interfaz grafica muy simple.

2 Requisitos Funcionales

- **RF-1:** Gestionar los clientes y camareros mediante hilos
- **RF-2:** Crear algun tipo de cola para que los clientes esperen por orden
- **RF-3:** Tiempo de espera de preparacion del café
- **RF-4:** Establecer un limite de tiempo segun cada cliente
- **RF-5:** Mostrar notificaciones de los distintos estados
- **RF-7:** Usar joins para finalizar los hilos, siendo este el punto mas importante.

3 Requisitos No Funcionales

- **RNF-1:** Evitar que un cliente no sea atendido por dos camareros al mismo tiempo
- **RNF-2:** Evitar que los camareros esten revisando en bucle, consumiendo muchos recursos
- **RNF-3:** Ajustar los tiempos de espera, entre tiempo de espera del cliente, pausas y preparacion para que no de errores
- **RNF-4:** Gestion de errores

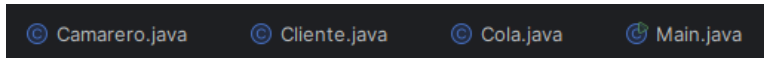
4 Historias de Usuario

- **HU-1:** Debo poder obtener un café en menos del tiempo que crea que sea excesivo.
- **HU-2:** Debo poder marcharme si tengo muy poca paciencia
- **HU-3:** Siendo Camarero, debe de ser el cliente el que me haga el pedido, no estare pendiente si hay clientes o no
- **HU-4:** Siendo Camarero atenderé al siguiente en la lista, no al que pretenda esperar menos.

5 Arquitectura

Dentro de este ejercicio tenemos varios elementos, un Cliente, Camarero y la Cola. El cliente y el camarero con threads y la cola es un elemento .java separado para administrar la cola de clientes.

Existen formas mas sencillas de administrar la cola como por ejemplo añadirlo al Camarero.java pero fue la forma mas clara de realizar el ejercicio.



Un elemento muy importante es que trate de que el cliente sea el que llama al camarero, intentando reducir el consumo de recursos al dejar al Camarero revisando cada milisegundo por un nuevo cliente.

```
//Si camarero no esta trabajando
if (cola.getClientesEnCola()==0) {
    activarCamarero();
}
//Si esta trabajando añadir a la cola sin mas
cola.agregarCliente(this);
```

6 Clases Principales

6.1 Clase Main

Main.class contiene los archivos fundamentales para inicar la aplicación y ejecutar cada uno de los módulos.

```
public class Main extends Application {
    public Main() {
    }
}
```

```

    public void start(Stage primaryStage) throws Exception {
        Parent root = (Parent)FXMLLoader.load(this.getClass().getResource("main.fxml"));
        Scene scene = new Scene(root, (double)720.0F, (double)480.0F);
        scene.getStylesheets().add(this.getClass().getResource("mainCSS.css").toURI());
        primaryStage.setResizable(false);
        primaryStage.setTitle("CafeteriaFX");
        primaryStage.setScene(scene);
        primaryStage.show();
    }

    public static void main(String[] args) {
        launch(args);
    }
}

```

6.2 Menu Controller

Esta se encarga de mapear las funciones de todos los elementos del javaFX, incluido el boton que da comienzo a la simulación

```

    public class MenuController {
        @FXML
        private Text colaClientesText;
        @FXML
        private Text estadoCamarerosText;
        @FXML
        private TextArea estadoCafeteriaText;
        @FXML
        private Button comenzarButton;
        private boolean simulacionEnCurso = false;

        public MenuController() {
        }

        @FXML
        public void initialize() {
            this.estadoCafeteriaText.setEditable(false);
            this.estadoCafeteriaText.setWrapText(true);
            this.comenzarButton.setOnAction((event) -> this.comenzarSimulacion());
        }

        private void comenzarSimulacion() {
            if (!this.simulacionEnCurso) {
                this.simulacionEnCurso = true;
                this.comenzarButton.setDisable(true);
            }
        }
    }

```

```

        this.colaClientesText.setText("");
        this.estadoCamarerosText.setText("");
        this.estadoCafeteriaText.setText("");
        Thread simulacionThread = new Thread(() -> {
            this.ejecutarSimulacionCompleta();
            Platform.runLater(() -> {
                this.comenzarButton.setDisable(false);
                this.simulacionEnCurso = false;
            });
        });
        simulacionThread.start();
    }
}

```

Tras ello cree una función "ejecutarSimulacionCompleta" que se encarga de todos los elementos fundamentales, añadir los clientes y camareros, iniciar estos y esperar a que terminen con un join()

```

private void ejecutarSimulacionCompleta() {
    Cola cola = new Cola();
    Camarero c1 = new Camarero("Camarero 1", cola, this);
    Camarero c2 = new Camarero("Camarero 2", cola, this);
    Camarero[] camareros = new Camarero[]{c1, c2};
    Random random = new Random();
    Cliente[] clientes = new Cliente[]{new Cliente("Ramon", random.nextInt(3),
        this.agregarMensajeCafeteria("Comienzo!"));

    for(int i = 0; i < clientes.length; ++i) {
        clientes[i].start();

        try {
            Thread.sleep((long)(random.nextInt(400) + 100));
        } catch (InterruptedException e) {
            e.printStackTrace();
        }
    }

    for(Cliente cliente : clientes) {
        try {
            cliente.join();
        } catch (InterruptedException e) {
            e.printStackTrace();
        }
    }

    try {
        Thread.sleep(2000L);
    }
}

```

```

    } catch (InterruptedException e) {
        e.printStackTrace();
    }

    this.agregarMensajeCafeteria("Todos los camareros terminaron! Fin");
}

```

Las siguientes funciones se encargan de actualizar los datos en los textareas de javaFX, como agregar clientes a la cola, eliminarlos, cambiar el estado del camarero y actualizar los mensajes del estado.

Al estar trabajando con hilos usé "Platform.runLater" para adaptarse a javaFX, debido a que esta solo permite modificar los datos desde su hilo principal.

```

private void ejecutarSimulacionCompleta() {
    Cola cola = new Cola();
    Camarero c1 = new Camarero("Camarero 1", cola, this);
    Camarero c2 = new Camarero("Camarero 2", cola, this);
    Camarero[] camareros = new Camarero[]{c1, c2};
    Random random = new Random();
    Cliente[] clientes = new Cliente[]{new Cliente("Ramon", random.nextInt(3000) + 7000, cola, camareros, this),
    new Cliente("Juanjo", random.nextInt(3000) + 5000, cola, camareros, this),
    new Cliente("Manuel", random.nextInt(5000) + 10000, cola, camareros, this),
    new Cliente("Ana", random.nextInt(2000) + 4000, cola, camareros, this)};
    this.agregarMensajeCafeteria("Comienzo!");

    for(int i = 0; i < clientes.length; ++i) {
        clientes[i].start();

        try {
            Thread.sleep((long)(random.nextInt(400) + 100));
        } catch (InterruptedException e) {
            e.printStackTrace();
        }
    }

    for(Cliente cliente : clientes) {
        try {
            cliente.join();
        } catch (InterruptedException e) {
            e.printStackTrace();
        }
    }

    try {
        Thread.sleep(2000L);
    } catch (InterruptedException e) {

```

```

        e.printStackTrace();
    }

    this.agregarMensajeCafeteria("Todos los camareros terminaron! Fin");
}

```

6.3 Camarero

Esta se centra en el camarero, ofreciendo todas los elemntos relaiconados con los procesos que tiene que realizar el hilo del camarero. Su funcion principal es "prepararCafe" la cual realiza todo el proceso una vez es activado por el cliente, siendo muy importantes todos los mensajes actualizando el estado de este.

6.4 Cliente

Cliente incluye todo lo relacionado con el, desde la creación de la class a la funcion de activarcamarero, fundamental para optimizar el programa. Tambien incluye todas las opciones desde notiifcar la entrada y su tiempo de espera a recibir el café o irse antes de tiempo

6.5 Cola

Esta class fue creada para organizarme correctamente y crear un codigo algo mas limpio, se centra en las funciones de agregar cliente a la cola,seleccionar el siguiente cliente en la cola, revisar si hay clientes en cola y contar el numero de clientes en ella.

7 Casos de uso

7.1 Desde el punto del Cliente al Camarero

El cliente entra en la cafetería y tiene un tiempo limite, este revisa el estado del camarero, si esta ocupado el cliente se une a la cola, si el camarero esta libre y no existe cola notifica al camarero.

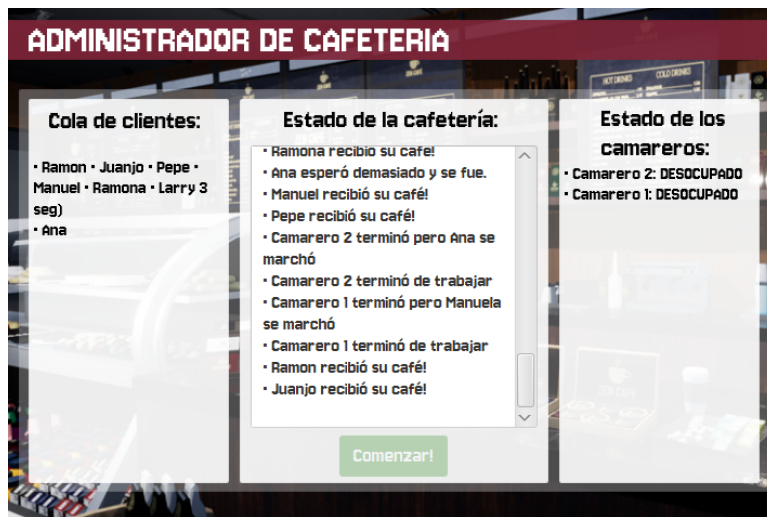
Una vez que termine la cola y se le asigne el cliente al camarero, este realizará el pedido en un tiempo breve pero aleatorio. Si pasa demasiado tiempo, incluso en la propia cola, el cliente se puede marchar, si el camarero esta preparando pero pasa demasiado tiempo, este se marcha sin mas.

Cuando todos los clientes son servidos o se marcharon, los camareros terminan de trabajar.

7.2 Desde el punto del usuario final del programa

El usuario simplemente tiene que hacer click en el boton verde y ver los resultados a tiempo real, estos son con elementos aleatorios por lo que cambian cada

vez que le das al boton, este no puede ser pulsado de nuevo hasta que el proceso termine.



8 Tecnologías usadas

Este proyecto fue realizado con Java, usando FXML, JavaFX para la interfaz gráfica principalmente para situar los elementos y tener una referencia visual. Use CSS debido a la cantidad de problemas que suelo tener con JavaFX, dandome cierto grado de seguridad y permitiendo usar fuentes de google mas interesantes, junto a imagenes de fondo.